



ENSIMAG – Grenoble INP
C++ pour les Mathématiques Appliquées

Simulation de Particules en C++

Rapport de projet – Construction progressive d’un simulateur

El Gouij Faical

Mountassir Hamza



Contents

1	Introduction	3
2	Guide utilisateur et organisation du dépôt	3
2.1	Prérequis	3
2.2	Compilation	3
2.3	Exécution des simulations	4
2.4	Visualisation	4
3	TP2 – Structures de base et premières trajectoires	5
4	TP3 – Univers de particules et potentiel de Lennard-Jones	6
4.1	Potentiel de Lennard-Jones	6
4.2	Intégration temporelle	6
5	TP4 – Décomposition spatiale et optimisation algorithmique	6
5.1	Principe de la grille de cellules	6
5.2	Simulation TP4 en 2D : collision carré–rectangle	7
5.3	Simulation 3D : collision cube–pavé	8
5.4	Validation par le suivi d’énergie	9
6	TP5 — Tests, visualisation et analyse du code	10
6.1	Infrastructure de tests	10
6.2	Compilation et exécution des tests	11
6.3	Visualisation scientifique	11
6.4	Analyse ACVL	11
7	TP6 – Raffinement du modèle : conditions aux limites, gravité et collisions	12
7.1	Conditions aux limites	12
7.2	Champ gravitationnel et collision boule–pavé	12
7.3	Contrôle de l’énergie cinétique	13
7.4	Limites de la validation énergétique au TP6}	14
8	Optimisations de performance	14
8.1	Contexte expérimental	14
8.2	Stratégie d’optimisation	14
9	Gestion mémoire : règle des cinq et RAI	16

10 Gestion des erreurs	16
11 Réponses aux questions du sujet	16
12 Conclusion	17

1 Introduction

Ce projet a pour objectif la construction progressive d'un simulateur de particules en C++. Au fil des travaux pratiques, le code évolue d'un prototype manipulant quelques objets vers un simulateur capable de gérer des systèmes de grande taille, avec interactions locales, visualisation scientifique, conditions aux limites, gravité et optimisations de performance.

Le modèle physique central repose sur le potentiel de Lennard-Jones, qui capture deux comportements essentiels : une répulsion intense à courte distance et une attraction de van der Waals à distance intermédiaire. L'évolution temporelle est assurée par le schéma de Störmer-Verlet, bien adapté aux systèmes mécaniques conservatifs en raison de sa meilleure stabilité par rapport aux intégrateurs explicites classiques.

2 Guide utilisateur et organisation du dépôt

Le dépôt est structuré comme un projet CMake, avec une séparation claire des rôles entre les répertoires :

- **include/** : en-têtes
- **src/** : implémentations et scripts de visualisation (**src/python_plot/**)
- **demo/** : exécutables de démonstration
- **test/** : tests unitaires
- **doc/** : documentation générée

2.1 Prérequis

Les dépendances nécessaires à la compilation et à l'exécution du projet sont les suivantes :

- Compilateur compatible **C++17** et **CMake** (version ≥ 3.20)
- **OpenMP** pour la parallélisation
- **Python 3** avec la bibliothèque *Matplotlib* (visualisation des résultats)
- **ParaView** pour la visualisation des fichiers VTK/VTU
- **Doxygen** pour la génération de la documentation
- **Google Test** pour les tests unitaires

2.2 Compilation

```
mkdir build && cd build && cmake .. && make -j
```

2.3 Exécution des simulations

Les exécutables sont générés dans `build/demo/` :

```
./demo/simu_tp2    ./demo/simu_tp4    ./demo/simu_tp6  
./demo/test_3d     ./demo/test_plot_ur
```

Depuis les modifications apportées à `univers.cxx` au TP6, la grande majorité des simulations exploite OpenMP automatiquement. Les seules exceptions sont `simu_tp2` et `test_plot_ur`, qui restent séquentielles. Le nombre de threads peut être contrôlé explicitement :

```
OMP_NUM_THREADS=4 ./demo/simu_tp4  
OMP_NUM_THREADS=4 ./demo/simu_tp6  
OMP_NUM_THREADS=4 ./demo/test_3d
```

Pour forcer une exécution séquentielle : `OMP_NUM_THREADS=1`.

Nous encourageons le lecteur à lancer les simulations de demo avec `OMP_NUM_THREADS=i` `./demo/simu`.

2.4 Visualisation

Les résultats peuvent être visualisés à l'aide des scripts Python fournis dans `src/python_plot/`, ou via ParaView pour les sorties au format VTK/VTU. Chaque script remplit un rôle précis :

- `plot_collision.py` : animation Matplotlib 2D de la collision ;
- `plot_collision_3d.py` : animation tridimensionnelle ;
- `plot_ur.py` : tracé du potentiel de Lennard-Jones $U(r)$;
- `plot_energie.py` : évolution temporelle des énergies, pour le contrôle de stabilité ;
- `script_visual.py` : trajectoires orbitales du TP2.

Ces scripts conviennent à une inspection rapide. Pour les simulations plus lourdes, le format VTK/VTU reste mieux adapté.

Les sorties de simulation sont organisées comme suit :

- **Frames de simulation**
 - `frames/`
 - `vtk_frames/`
 - `vtu_frames/`
- **Données énergétiques et potentiel en csv**
 - `energy/`
 - `potential/`

Le chemin exact de sortie est affiché dans le terminal lors de l'exécution.

La documentation Doxygen est générée depuis le répertoire `build/` avec la commande :

```
make doc
```

3 TP2 – Structures de base et premières trajectoires

Le TP2 constitue la première étape du projet : mettre en place les briques fondamentales de la simulation. La classe `vecteur` représente positions, vitesses et forces dans un espace tridimensionnel, y compris pour des simulations en dimension inférieure. La classe `particule` encapsule l'état physique complet d'un objet : masse, position, vitesse, force et type. Cette séparation distingue clairement les données physiques locales de la logique globale de simulation, laquelle est portée par la classe `univers`.

Une première validation qualitative consiste à simuler un système orbital simplifié, comprenant le Soleil, la Terre, Jupiter et la comète de Halley. L'objectif est de vérifier que les trajectoires sont physiquement cohérentes et que le schéma d'intégration produit un comportement stable sur plusieurs révolutions.

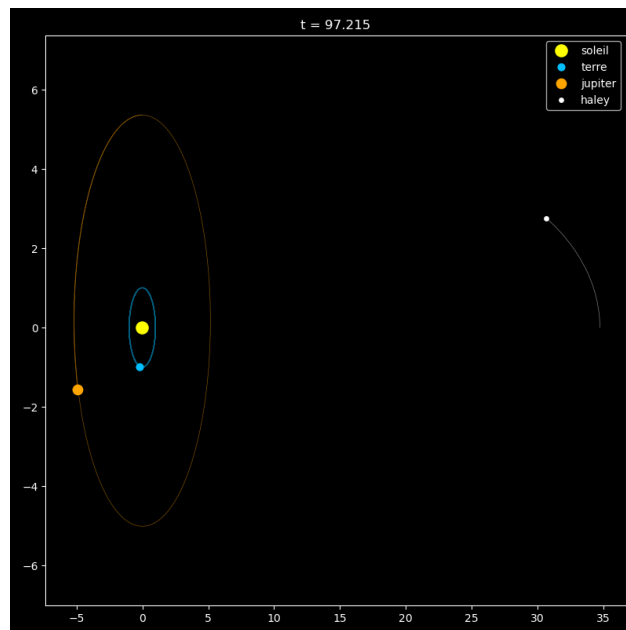


Figure 1: Simulation orbitale TP2 : trajectoires du Soleil, de la Terre, de Jupiter et de la comète de Halley.

4 TP3 – Univers de particules et potentiel de Lennard-Jones

Le TP3 introduit le cœur physique du simulateur. La classe `univers` devient l'objet central, responsable de la gestion des particules, du calcul des forces d'interaction et de l'orchestration de l'évolution temporelle.

4.1 Potentiel de Lennard-Jones

L'interaction entre deux particules séparées par une distance r est modélisée par :

$$U_{LJ}(r) = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad \mathbf{F}_{ij} = \frac{24\varepsilon}{r_{ij}^2} \left[2 \left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right] \mathbf{r}_{ij}.$$

Le terme en r^{-12} modélise la répulsion à très courte portée ; le terme en r^{-6} représente l'attraction de van der Waals. Les interactions sont tronquées à $r_{\text{cut}} = 2.5\sigma$, préfigurant l'optimisation par grille introduite au TP4.

4.2 Intégration temporelle

L'évolution temporelle repose sur le schéma de Störmer-Verlet :

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t) + \frac{\Delta t^2}{2m_i} \mathbf{F}_i(t), \quad \mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \frac{\Delta t}{2m_i} [\mathbf{F}_i(t) + \mathbf{F}_i(t + \Delta t)].$$

Ce schéma symplectique sépare naturellement la mise à jour des positions et des vitesses. Il nécessite de conserver les forces au pas précédent et de recalculer les forces après déplacement. La classe `univers` gère l'ensemble de ces opérations – liste des particules, calcul des forces, pas de temps et grandeurs globales comme l'énergie – ce qui la rend facilement extensible pour les travaux pratiques suivants.

5 TP4 – Décomposition spatiale et optimisation algorithmique

Dans une implémentation naïve, le nombre de paires à évaluer est $N(N-1)/2$, soit une complexité $\mathcal{O}(N^2)$ qui devient rapidement rédhibitoire dès que N croît. Le TP4 remédie à cela en introduisant une décomposition spatiale par grille de cellules.

5.1 Principe de la grille de cellules

L'espace de simulation est découpé en cellules régulières de côté fondé sur r_{cut} . Le nombre de cellules dans la direction k est $n_k = \lfloor L_k / r_{\text{cut}} \rfloor$. Chaque particule n'interagit qu'avec les particules appartenant à sa propre cellule ou aux cellules immédiatement voisines, ramenant la complexité effective à $\mathcal{O}(N)$ à densité bornée.

Plusieurs optimisations locales complètent cette structure : comparaison des distances au carré pour éviter les appels à `sqrt`, multiplications successives pour s'affranchir de `pow`, et exploitation du

principe d'action-réaction pour ne calculer chaque interaction qu'une seule fois. Appliquées dans la boucle la plus coûteuse du simulateur, elles produisent un gain global significatif.

5.2 Simulation TP4 en 2D : collision carré–rectangle

La première expérience de collision est réalisée avec l'exécutable `simu_tp4`. Elle met en interaction un carré de particules en chute verticale avec un rectangle de particules initialement immobile. Cette simulation sert principalement à valider le passage à l'échelle obtenu par la grille de cellules, tout en conservant un scénario suffisamment simple à interpréter visuellement.

Certains éléments du modèle physique ne sont volontairement pas détaillés ici, notamment la liste de Verlet ainsi que les conditions aux limites. Ces aspects seront présentés plus en détail dans les sections ultérieures, en particulier lors du TP6 pour les conditions aux limites, et dans la partie consacrée aux optimisations pour la structure de voisinage. Dans ce cas précis du TP4, aucune condition aux limites n'est activée, ce qui permet de se concentrer uniquement sur les interactions internes du système.

Table 1: Paramètres détaillés de la simulation TP4 en 2D.

Paramètre	Valeur
Exécutable	<code>simu_tp4</code>
Dimension	2D
Domaine	$L1 = 250, L2 = 150$
Paramètres Lennard–Jones	$\sigma = 1, \epsilon = 5, \text{masse} = 1$
Distance de coupure	$r_{\text{cut}} = 2.5 \sigma$
Pas de temps	$\Delta t = 5e-5$
Durée simulée	$T = 19.5$
Nombre d'itérations	390000 pas
Distance initiale entre particules	$2^{1/6} / \sigma$
Objet supérieur	40 x 40 particules, soit $N1 = 1600$
Objet inférieur	160 x 40 particules, soit $N2 = 6400$
Nombre total de particules	$N = 8000$
Vitesses initiales	bloc supérieur : $(0, -10, 0)$; bloc inférieur : $(0, 0, 0)$
Conditions aux limites	aucune condition aux limites activée
Liste de Verlet	activée, avec un skin de 0.3
Sauvegarde	une frame toutes les 1000 itérations
Suivi énergétique	écriture de <code>energy/energie_tp4.csv</code> toutes les 1000 itérations

Cette expérience met en évidence le rôle déterminant de la décomposition spatiale. Sans grille de cellules, une simulation contenant 8000 particules impliquerait un nombre de paires trop élevé pour rester exploitable. Avec la grille, seules les interactions locales sont évaluées, ce qui rend possible

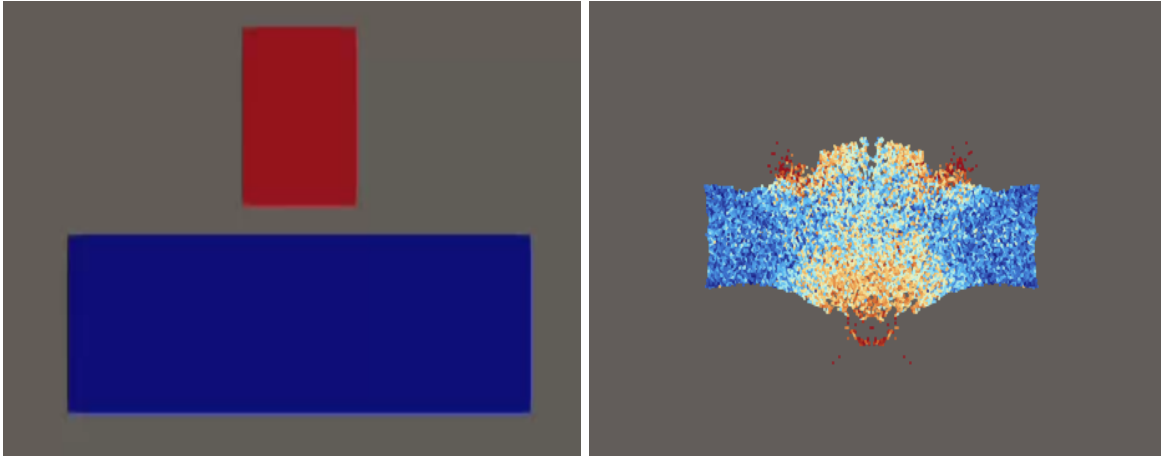


Figure 2: Simulation TP4 : état initial et dispersion des structures de particules après collision.

une simulation longue avec sauvegarde régulière des frames.

5.3 Simulation 3D : collision cube–pavé

Une seconde expérience, absente de la première version du rapport, est fournie par l'exécutable `test_3d`. Elle reprend la logique de collision du TP4, mais dans un domaine tridimensionnel. L'objectif est de vérifier que les classes `vecteur`, `particule`, `univers` et les exports VTK/VTU restent cohérents lorsque les trois composantes spatiales sont réellement utilisées.

Table 2: Paramètres détaillés de la simulation tridimensionnelle `test_3d`.

Paramètre	Valeur
Exécutable	<code>test_3d</code>
Dimension	3D
Domaine	$L1 = 120, L2 = 120, L3 = 120$
Paramètres Lennard–Jones	$\sigma = 1, \epsilon = 5, \text{masse} = 1$
Distance de coupure	$r_{\text{cut}} = 2.5 \sigma$
Pas de temps	$\Delta t = 1e-4$
Durée simulée	$T = 10$
Nombre d'itérations	100000 pas
Distance initiale entre particules	$2^{(1/6)} / \sigma$
Objet supérieur	cube $10 \times 10 \times 10$, soit $N1 = 1000$
Objet inférieur	pavé $10 \times 30 \times 10$, soit $N2 = 3000$
Nombre total de particules	$N = 4000$
Vitesses initiales	cube : $(0, 0, -8)$; pavé : $(0, 0, 0)$
Position du pavé	$x0 = 20, y0 = 20, z0 = 15$

Paramètre	Valeur
Position du cube	centré au-dessus du pavé, avec un décalage vertical de 5
Sauvegarde	une frame toutes les 200 itérations
Formats de sortie	texte, VTK legacy ou VTU XML

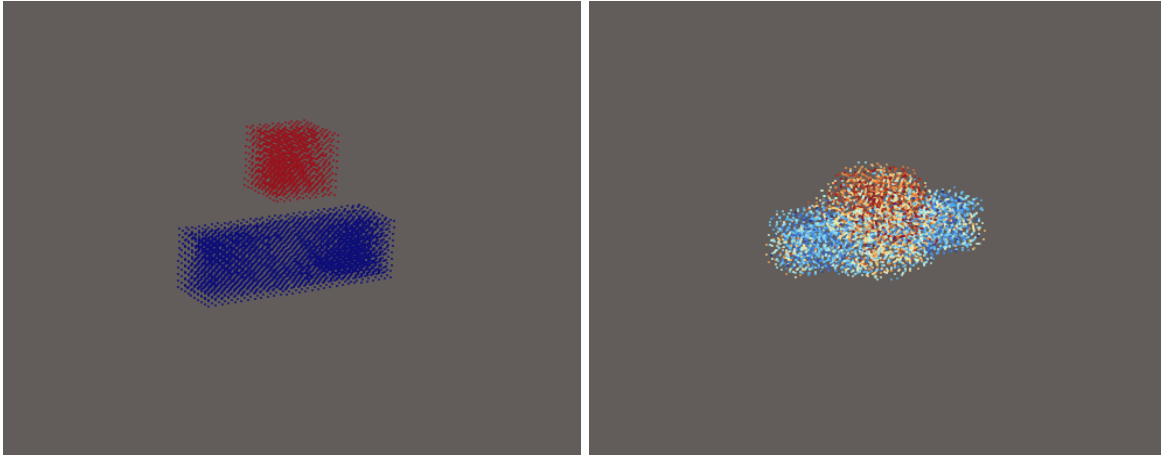


Figure 3: Simulation TP4 : état initial et dispersion des structures de particules après collision.

Cette simulation joue un rôle complémentaire par rapport au cas 2D : elle ne vise pas seulement la performance, mais aussi la validation géométrique du simulateur en dimension trois. Elle permet notamment de vérifier l'export des coordonnées (x, y, z) et l'exploitation des fichiers `.vtk.series` ou `.vtu.series` pour l'animation sous ParaView.

Cette étape illustre l'idée directrice du projet : les gains de performance les plus décisifs proviennent du choix de l'algorithme. Le passage à la grille de cellules constitue, de loin, l'amélioration la plus structurante pour simuler des systèmes de grande taille.

5.4 Validation par le suivi d'énergie

Un suivi des énergies a été ajouté lors des simulations du TP4. Les valeurs de l'énergie cinétique, potentielle et mécanique totale sont enregistrées dans le répertoire `energy/`, puis visualisées à l'aide de `plot_energie.py`.

On observe que l'énergie cinétique et l'énergie potentielle présentent de fortes variations au cours de la collision, ce qui est physiquement attendu : l'énergie est continuellement échangée entre ces deux formes lors des interactions entre particules.

En revanche, l'énergie mécanique totale $\mathbf{E_m}$ (somme des énergies cinétique et potentielle) se conserve globalement au cours du temps, à de faibles fluctuations près. Ces écarts sont dus aux erreurs numériques inhérentes à la discrétisation temporelle (schéma de Verlet) ainsi qu'aux approximations de calcul.

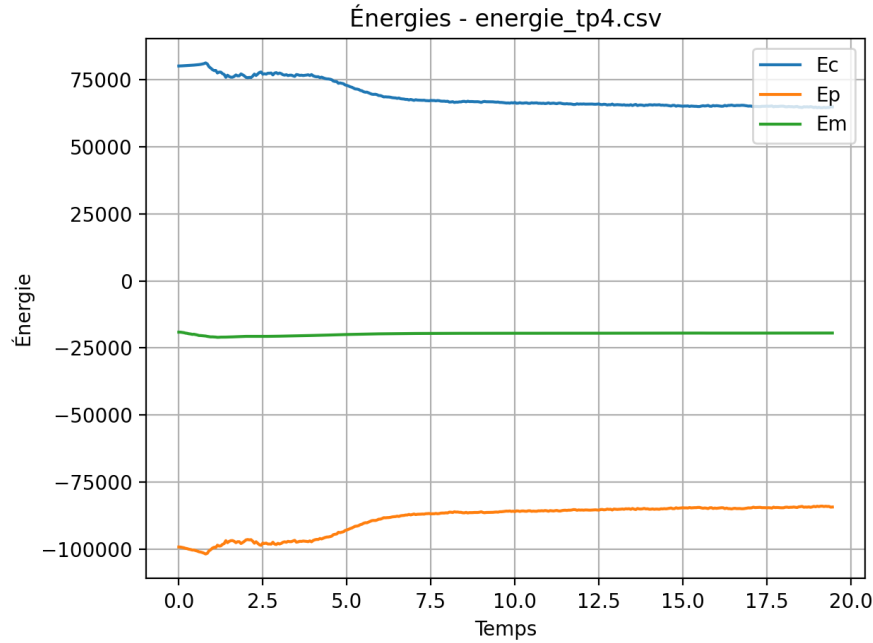


Figure 4: Évolution des énergies cinétique, potentielle et mécanique totale au cours d’une simulation TP4.

Cette quasi-conservation constitue un indicateur essentiel de la validité de la simulation. En effet, dans le cadre du TP4, les seules forces mises en jeu dérivent du potentiel de Lennard–Jones, qui est un potentiel conservatif. Par conséquent, l’énergie mécanique du système doit être conservée en théorie.

Le fait que $\mathbf{E}_m$ reste constante à de faibles erreurs près confirme à la fois la cohérence du schéma d’intégration utilisé et la validité du calcul des interactions entre particules.

6 TP5 — Tests, visualisation et analyse du code

6.1 Infrastructure de tests

Le TP5 introduit l’utilisation de Google Test afin de vérifier le bon fonctionnement des principales classes du projet : `vecteur`, `particule`, `cellule` et `univers`.

Les tests sont organisés selon une hiérarchie :

- **Tests unitaires bas niveau** : vérification des opérations vectorielles, constructeurs et méthodes simples.
- **Tests intermédiaires** : validation des interactions entre particules et cellules.
- **Tests système** : contrôle de la cohérence globale de la grille et des comportements physiques

au cours du temps.

Cette infrastructure permet à la fois : - la détection de régressions lors des modifications du code, - la documentation du comportement attendu de chaque composant.

6.2 Compilation et exécution des tests

Les tests sont compilés et exécutés via CMake et CTest.

```
mkdir -p build
cd build
cmake ..
make
ctest --output-on-failure
```

6.3 Visualisation scientifique

Le TP5 étend les outils de visualisation introduits précédemment. En effet, le format VTK avait déjà été utilisé au TP4 pour exporter des états du système et permettre leur visualisation avec ParaView.

Le TP5 introduit plus spécifiquement le format VTU, qui correspond à une version XML structurée du format VTK. Ce format est mieux adapté à la manipulation de données complexes et volumineuses, et facilite l'intégration avec les outils modernes de visualisation.

Les fichiers exportés contiennent notamment les positions, vitesses, masses et autres données associées à chaque particule.

Lorsque le nombre de particules augmente, les fichiers texte et les animations Matplotlib deviennent rapidement inadaptés en termes de performance et de lisibilité. L'utilisation des formats VTK/VTU permet alors une visualisation plus efficace et plus scalable des systèmes de grande taille, en s'appuyant sur les capacités avancées de ParaView.

6.4 Analyse ACVL

La dernière partie du TP5 consiste en une analyse architecturale a posteriori : diagramme de cas d'utilisation, de séquence, de transitions et de classes. Cette étape oblige à prendre du recul sur la structure du simulateur, qui devient un système bien défini avec des responsabilités identifiées et des interactions clairement formalisées entre composants. Les diagrammes produits et les scripts **PUML** utilisés pour les générer sont disponibles dans `diagrammes/doc/diagrammes/[type_du_diagramme]`, avec un accès direct aux fichiers **png** correspondants dans `TP6/diagrammes_uml/`.

Ces diagrammes ne couvrent pas l'intégralité des méthodes, classes et fonctionnalités du projet. Nous avons volontairement sélectionné les éléments les plus structurants et représentatifs du fonctionnement global du simulateur. En effet, en raison de la taille du projet, de nombreuses méthodes supplémentaires, notamment celles liées aux optimisations et à l'implémentation bas niveau, n'ont pas été incluses, car elles n'interviennent pas directement dans la logique fonctionnelle principale

mais plutôt dans des améliorations de performance ou des détails d’implémentation.

7 TP6 – Raffinement du modèle : conditions aux limites, gravité et collisions

7.1 Conditions aux limites

Trois types de conditions aux limites sont implémentés. La condition **réflexive** renvoie la particule dans le domaine en inversant la composante de vitesse normale à la paroi ou, dans une variante plus progressive, par un potentiel de mur :

$$F_{\text{mur}}(r) = -24\varepsilon \cdot \frac{1}{2r} \left(\frac{\sigma}{2r} \right)^6 \left[1 - 2 \left(\frac{\sigma}{2r} \right)^6 \right], \quad r_{\text{cut}}^{\text{mur}} = 2^{1/6} \sigma.$$

Cette formulation évite la discontinuité brusque de l’inversion directe et produit une interaction plus régulière avec le bord. La condition **absorbante** supprime toute particule sortant du domaine. La condition **périodique** la fait réapparaître du côté opposé, simulant un domaine infini par translation.

7.2 Champ gravitationnel et collision boule–pavé

Le TP6 ajoute un champ gravitationnel uniforme $\mathbf{F}_i^G = (0, m_i G)$ avec $G < 0$, qui s’additionne aux forces de Lennard–Jones à chaque pas de temps. Le scénario principal fait tomber une boule de particules sur un pavé massif. Il s’agit du cas le plus exigeant du projet, car il combine grand nombre de particules, gravité, collisions violentes, conditions aux limites et limitation de l’énergie cinétique.

Table 3: Paramètres détaillés de la simulation TP6 boule–pavé.

Paramètre	Valeur
Exécutable	<code>simu_tp6</code>
Dimension	2D
Domaine	L1 = 250, L2 = 180
Paramètres Lennard–Jones	sigma = 1, epsilon = 1, masse boule = 2, masse pavé = 2.001
Distance de coupure	r_cut = 2.5 sigma
Gravité	G = -12
Pas de temps	Delta t = 5e-4
Durée simulée	T = 29.5
Nombre d’itérations	59000 pas
Distance initiale entre particules	2^(1/6) sigma
Pavé inférieur	N2x = 179, N2y = 96, soit N2 = 17184 particules

Paramètre	Valeur
Boule supérieure	$N_1 = 395$ particules, choisies parmi les points les plus proches du centre d'un disque de recherche
Nombre total de particules	$N = 17579$
Vitesses initiales	boule : $(0, -10, 0)$; pavé : $(0, 0, 0)$
Conditions aux limites	choisies à l'exécution sur $x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$; réflexives par défaut
Potentiel de mur	activé
Liste de Verlet	activée, avec un skin de 0.5
Limiteur de vitesse	activé à partir de l'itération 6000, puis toutes les 1000 itérations
Sauvegarde	une frame toutes les 100 itérations
Suivi énergétique	prévu dans <code>energy/energie_tp6.csv</code> ; dans le code actuel, l'écriture détaillée est désactivée dans le bloc de debug

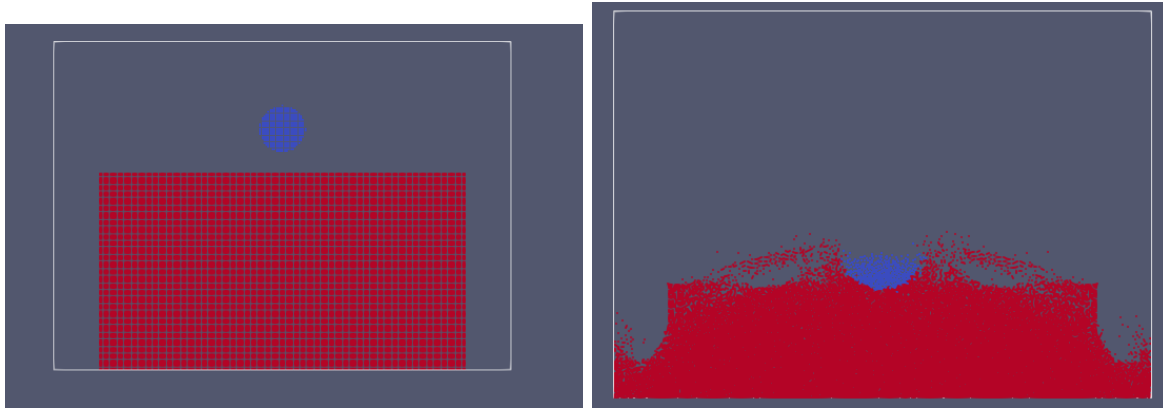


Figure 5: Simulation TP6 : collision sous gravité entre une boule et un pavé de particules.

Cette simulation est plus difficile à stabiliser que le cas TP4. La gravité accélère la boule avant l'impact, les interactions de Lennard–Jones deviennent très fortes au contact, et le pavé contient un grand nombre de particules. Le choix d'un pas de temps plus petit, l'utilisation de la liste de Verlet et le limiteur d'énergie cinétique sont donc nécessaires pour garder une évolution exploitable.

7.3 Contrôle de l'énergie cinétique

Lors de l'impact, certaines particules peuvent acquérir des vitesses très élevées, menaçant la stabilité numérique de la simulation. Pour limiter cette divergence, les vitesses sont périodiquement renormalisées par un facteur $\beta = \sqrt{E_c^D / E_c}$, où $E_c^D = 0.005(N_1 + N_2)$. Cette méthode ne conserve pas exactement l'énergie totale, mais elle maintient la simulation dans un état exploitable face à un scénario physiquement très violent.

7.4 Limites de la validation énergétique au TP6}

Dans le TP6, l'énergie mécanique totale ne constitue plus un critère fiable de validation, contrairement au TP4. Cela s'explique par l'introduction de plusieurs éléments qui perturbent sa conservation : les conditions aux limites (réflexives ou absorbantes), les collisions avec les parois et le contrôle artificiel de l'énergie cinétique.

Ces mécanismes ajoutent des effets non conservatifs ou numériques qui modifient l'énergie du système indépendamment de la physique pure. Ainsi, l'évolution de l'énergie reflète autant les choix algorithmiques que le comportement physique.

Dans ce contexte, la validation repose davantage sur l'observation qualitative des trajectoires et des structures formées que sur la conservation de l'énergie.

8 Optimisations de performance

8.1 Contexte expérimental

Les mesures de performance ont été réalisées sur une machine équipée d'un processeur **Intel Core i5 de 7ème génération (4 cœurs physiques)**. Toutes les performances rapportées après optimisation correspondent à une exécution avec OpenMP sur 4 threads, qui constitue le meilleur compromis observé.

Table 4: Comparaison des performances avant et après optimisation

Simulation	Threads_OMP	Avant_optimisation	Après_optimisation	Gain_estime
TP4 - 2D	4 threads	1268 s	201 s	$\sim 6.34\times$
TP4 - 3D	4 threads	non communiqué	135 s	non calculable
TP6	4 threads	$\sim 1000\text{--}1100$ s	220 s	$\sim 4.5\text{--}5\times$

8.2 Stratégie d'optimisation

Les optimisations introduites dans le projet peuvent être regroupées en plusieurs catégories, résumées dans le tableau suivant.

Catégorie	Explication
Parallélisation	Parallélisation OpenMP : la simulation est répartie sur 4 threads. Chaque thread traite un sous-ensemble de particules avec des buffers privés afin d'éviter les conflits mémoire et de maximiser l'utilisation des cœurs CPU.
Boucle critique	Boucle critique optimisée : suppression des opérations coûteuses comme <code>pow()</code> . Les distances sont calculées avec des multiplications directes ($dx*dx + dy*dy$), ce qui réduit fortement le coût de calcul.
Structure de données	Grille spatiale (spatial hashing) : l'espace est découpé en cellules. Les interactions ne sont calculées qu'entre cellules voisines et uniquement les cellules occupées. En pratique, la majorité des cellules sont vides, donc on ne parcourt que les cellules réellement occupées, ce qui évite un balayage inutile de toute la grille et réduit fortement la complexité.
Mémoire	Optimisation mémoire : utilisation de <code>reserve()</code> pour éviter les reallocations dynamiques et passage des structures par référence pour éviter les copies inutiles.
Compilation	Optimisation compilation (CMake) : compilation avec <code>-O3</code> (optimisation maximale), <code>-march=native</code> (instructions CPU SIMD), <code>-funroll-loops</code> (dépliage des boucles), <code>-ffast-math</code> (optimisations flottantes), <code>-flto</code> (optimisation inter-fichiers), et <code>-DNDEBUG</code> (désactivation des assertions).
Algorithme physique	Action-réaction : chaque interaction est calculée une seule fois ($F_{ij} = -F_{ji}$), divisant le nombre de calculs de forces par deux.
Optimisation avancée	Liste de Verlet : chaque particule maintient une liste de voisins dans un rayon élargi (skin distance). Grâce au petit pas de temps, les particules se déplacent peu entre deux frames, ce qui permet de réutiliser la liste sur plusieurs itérations. Cette optimisation est très efficace pour des petits pas de temps, mais devient moins stable si le pas est grand. Le choix du skin introduit un compromis entre fréquence de mise à jour et coût de calcul.

9 Gestion mémoire : règle des cinq et RAII

La classe `univers` est propriétaire des particules qu'elle contient. Les cellules ne stockent que des pointeurs non propriétaires : vider une cellule ne doit en aucun cas détruire les particules sous-jacentes. Comme `univers` gère des ressources dynamiques, la règle des cinq est rigoureusement respectée :

- **Destructeur** : libère toutes les particules allouées dynamiquement.
- **Constructeur de copie** : effectue une copie profonde pour éviter que deux univers partagent les mêmes particules.
- **Opérateur d'affectation par copie** : libère l'ancien contenu avant de copier le nouvel état.
- **Constructeur de déplacement** : transfère efficacement la propriété des ressources sans copie, en vidant la source.
- **Opérateur d'affectation par déplacement** : libère les ressources existantes, puis transfère celles de la source.

Le déplacement est particulièrement utile lorsque le système contient plusieurs milliers de particules : il transfère la propriété en $\mathcal{O}(1)$, sans aucune allocation supplémentaire. L'ensemble de ces mécanismes garantit également la libération correcte des ressources lors du déroulement de pile en cas d'exception, conformément au principe RAII.

10 Gestion des erreurs

Le TP6 introduit un mécanisme structuré de gestion des erreurs par exceptions, regroupées en trois catégories :

- **Erreurs de paramètres** : dimension invalide, pas de temps négatif, domaine incohérent
- **Erreurs de fichiers** : écriture ou création impossible
- **Erreurs numériques** : apparition de NaN, énergie divergente

L'usage des exceptions permet de produire des messages d'erreur explicites et de terminer proprement la simulation, plutôt que de continuer avec un état invalide dont les résultats seraient inexploitable.

11 Réponses aux questions du sujet

Q.10 – TP3. Les optimisations algorithmiques principales portent sur la réduction du coût des interactions : grille de cellules pour éliminer le coût quadratique, comparaison des distances au carré pour éviter les `sqrt`, remplacement de `pow` par des multiplications successives, réduction des

allocations et des copies inutiles, et exploitation du principe d'action-réaction.

Q.4 – TP6. Le TP6 introduit gravité et collisions à grande échelle. Les principaux défis rencontrés sont la stabilité numérique lors des impacts, le coût du calcul des forces dans les systèmes denses, la gestion mémoire avec un grand nombre de particules, et le volume des fichiers générés. Les solutions mises en place ont été détaillées dans les sections précédentes.

12 Conclusion

Ce projet a permis de construire, étape par étape, un simulateur de particules en C++ moderne. Chaque travail pratique apporte une dimension nouvelle : structures de base, interactions physiques, décomposition spatiale, tests, visualisation, conditions aux limites, gravité, collisions, parallélisation et robustesse.

La grille de cellules constitue l'amélioration algorithmique la plus déterminante : elle permet de passer d'une complexité quadratique à une complexité quasi-linéaire, rendant les grands systèmes simulables en un temps raisonnable. Les optimisations locales et la parallélisation OpenMP apportent des gains supplémentaires notables, mais ils n'auraient pas suffi sans ce changement de paradigme algorithmique.